All images loaded

Ficha 10 (26) - CI/CD

Ficha 10: setup e deploy com CI/CD

Nesta ficha faremos duas coisas:

- treino de 30 min de modelação, para consolidação
- setup e deploy da aplicacao portfolio com CI/CD.

Ficarão com a vossa aplicação em produção pública na internet

A. Quizz de Modelação

30 min para resolver este problema

Há várias organizações que promovem atividades. Uma atividade é organizada por uma organização e tem uma data e local. Cada voluntário tem um perfil com dados de contacto e interesses. Um voluntário pode participar em várias atividades, inscrevendo-se. As organizações podem dar um feedback ao voluntário após a atividade.

Para cada cenário:

- **Descreva as entidades e relações, e esboce o Diagrama Entidade-Relação (DER)**
- **construa as classes correspondentes em `models.py`, de acordo com a estrutura de dados definida**
- **Desenhe num papel duas visualizações diferentes (layouts de interface no browser) para apresentar os dados.**
- **Para cada visualização, escreva no papel uma função para `views.py` e o respetivo template HTML que apresente os conteúdos de acordo com o layout definido.**

B. Arquitetura de Web Apps Modernas

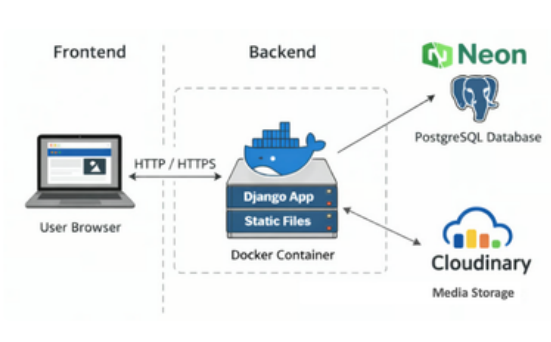
Separação entre Aplicação, Dados e Storage

O que é uma Web App moderna?

Uma aplicação web moderna:

- não guarda dados nem ficheiros no servidor da aplicação
- usa serviços externos para persistência (dados e media)
- é modular, distribuída e escalável

Arquitetura


Frontend
Backend
Neon
PostgreSQL Database
HTTP / HTTPS
User Browser
Django App
Static Files
Docker Container
Cloudinary
Media Storage

Regra de ouro

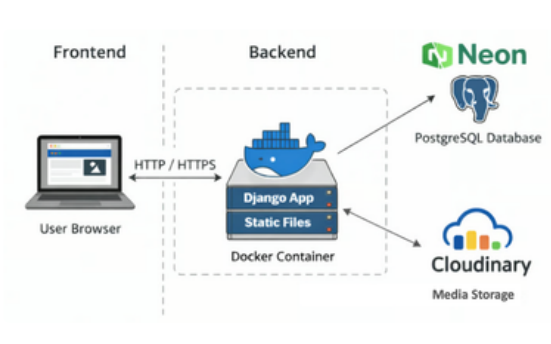
Separar responsabilidades entre componentes

🧠 **Aplicação (lógica)** → Django (em container)

📦 **Dados** → Base de dados (PostgreSQL, em Neon)

🖼️ **Ficheiros** → Storage de media (Cloudinary)

Componentes


Frontend
Backend
Neon
PostgreSQL Database
HTTP / HTTPS
User Browser
Django App
Static Files
Docker Container
Cloudinary
Media Storage

🐳 **Aplicação Django**

👉 **o cérebro que liga tudo**

- recebe pedidos do browser
- gestão das páginas e executa a lógica da aplicação
- comunica com a base de dados e com o storage de media

🐘 **Base de dados**

👉 **persistência de dados estruturados**

- guarda dados dinâmicos (tabelas da base de dados)
- não guarda ficheiros multimédia (imagens, vídeos, PDFs)
- serviço externo (e.g., Neon)

☁️ **Storage de ficheiros (Media)**

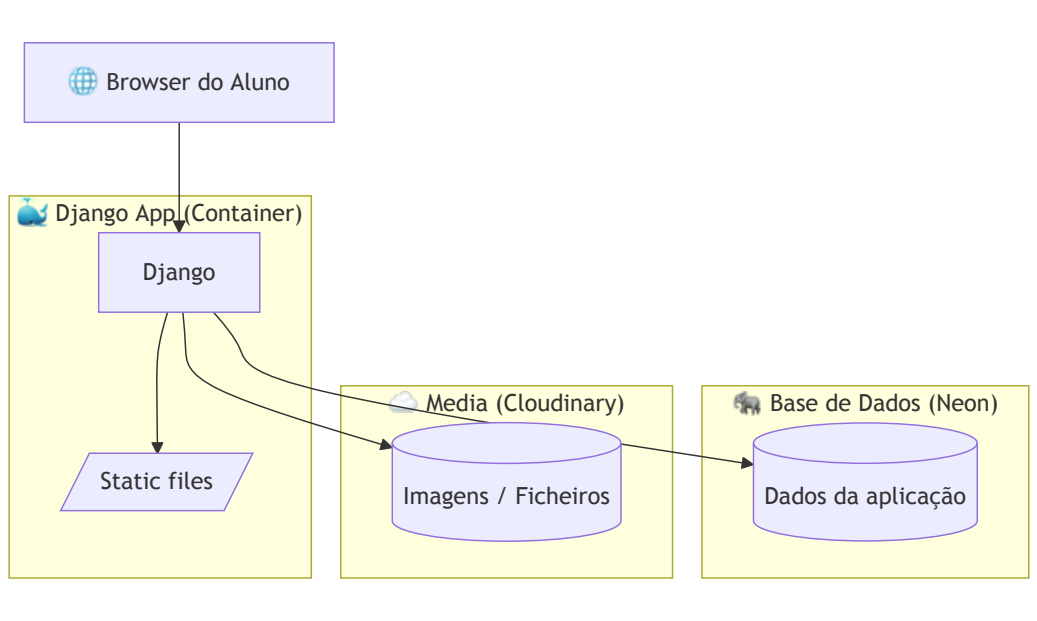
👉 **ficheiros enviados pelos utilizadores** (uploads)

- serviço externo (e.g., Cloudinary)
- acesso rápido via internet
- não ocupa espaço no servidor

🧊 📁 **Static files**

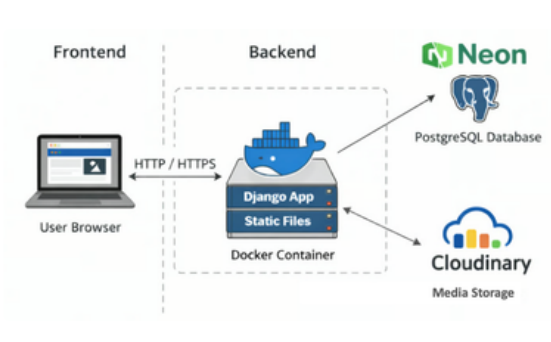
👉 **recursos fixos do site** (CSS, JS, imagens do layout)

- não vêm da base de dados
- não so enviados pelos utilizadores
- são carregados pelo browser quando uma página é aberta
- podem ser servidos por CDN para melhor performance

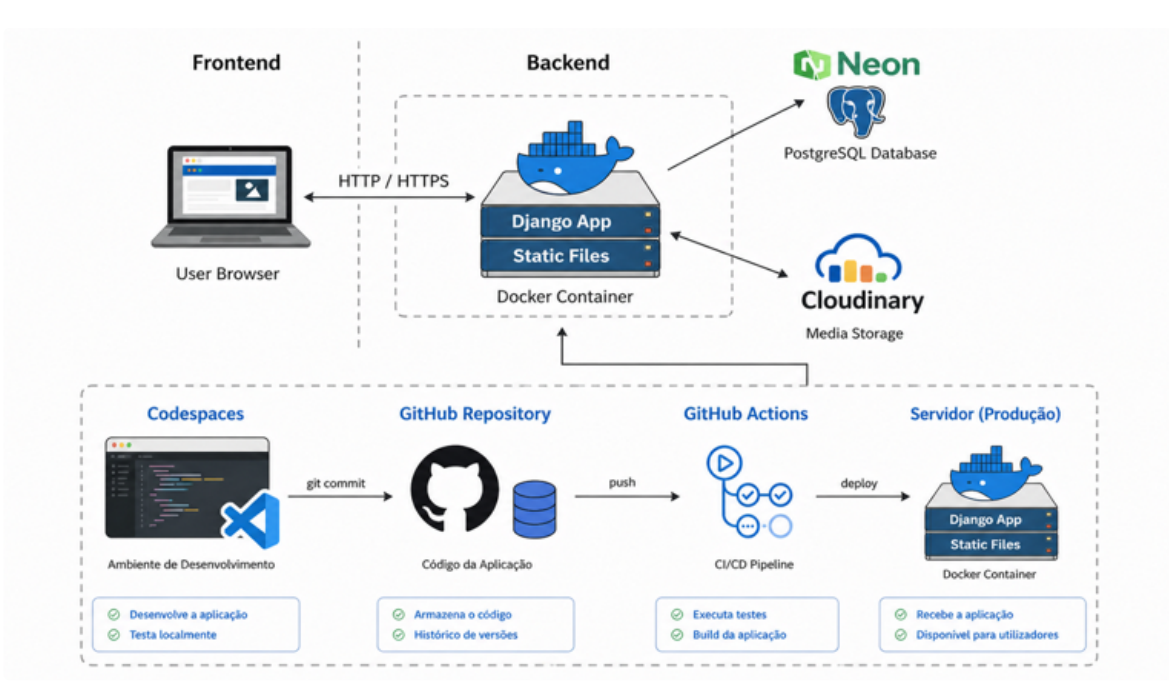

🌐 Browser do Aluno
🐳 Django App (Container)
Django
Static files
☁️ Media (Cloudinary)
Imagens / Ficheiros
🐘 Base de Dados (Neon)
Dados da aplicação

Fluxos

- O **browser** faz pedidos (*requests*) HTTP ao Django
- O **Django** processa tudo
- Os **dados** vão para/vêem da base de dados (Neon)
- Os **ficheiros/imagens** vão para Cloudinary
- Os **ficheiros estáticos** fazem parte da app


Frontend
Backend
Neon
PostgreSQL Database
HTTP / HTTPS
User Browser
Django App
Static Files
Docker Container
Cloudinary
Media Storage

Arquitetura e Pipeline de Deploy de Aplicação Web


Frontend
Backend
Neon
PostgreSQL Database
HTTP / HTTPS
User Browser
Django App
Static Files
Docker Container
Cloudinary
Media Storage
Codespaces
Ambiente de Desenvolvimento
Desenvolve a aplicação
Testa localmente
git commit
GitHub Repository
Código da Aplicação
Armazena o código
Histórico de versões
push
GitHub Actions
CI/CD Pipeline
Executa testes
Build da aplicação
deploy
Servidor (Produção)
Django App
Static Files
Docker Container
Recebe a aplicação
Disponível para utilizadores

Neste laboratório iremos implementar esta arquitetura, incluindo uma pipeline de deploy.

CI/CD: Integração e Entrega Contínua:

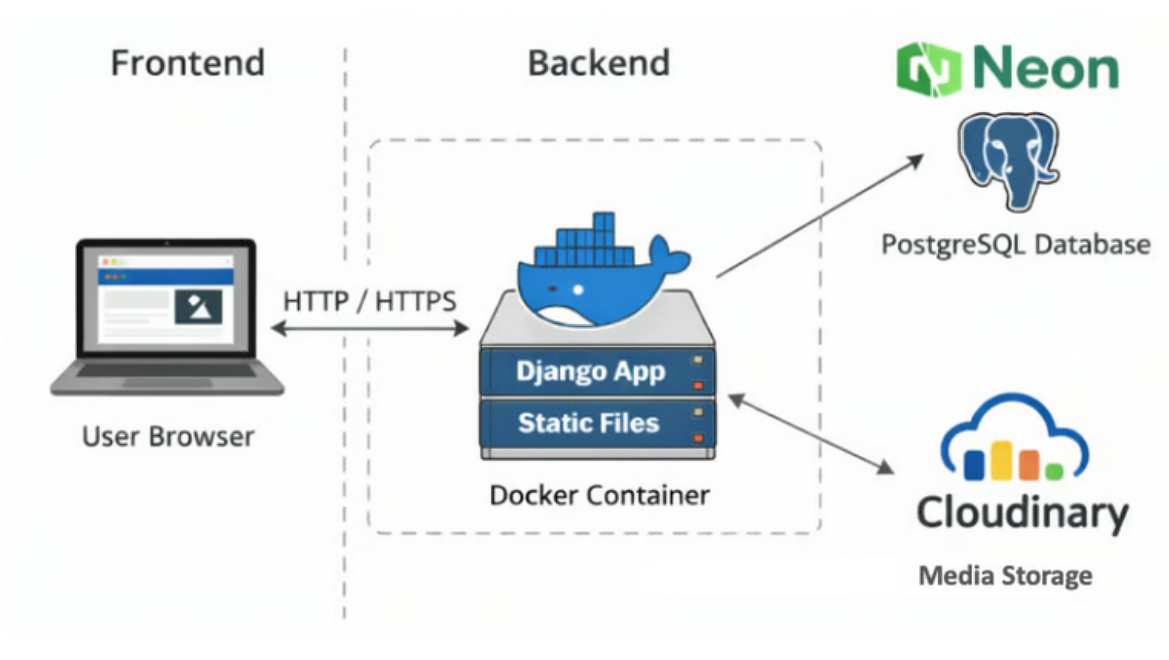
- No Codespaces ou localmente no nosso PC desenvolvemos a aplicação (ambiente de desenvolvimento)
- As alterações são enviadas ao GitHub, que desencadeia um processo automático de validação, teste e deploy no servidor (ambiente de produção)

Arquitetura do seu portfólio

Como referido, neste lab irá expandir o seu projeto de portfolio numa arquitetura como em baixo.

Tem várias partes:

1. usar uma base de dados PostgreSQL, alojada no Neon
2. usar um storage de ficheiros, alojados no Cloudinary
3. alojamento da sua aplicação num servidor da lusófona


Frontend
Backend
Neon
PostgreSQL Database
HTTP / HTTPS
User Browser
Django App
Static Files
Docker Container
Cloudinary
Media Storage

Os três passos sao independentes. Terá sempre a sua aplicação a funcionar ao longo da sua implementação.

0. Preparativos

Ambiente virtual e dependências

1. **Codespaces**

   Abra o seu repositório do portfolio no codespaces

2. **Criação e ativação de ambiente virtual**

   Crie um ambiente virtual com o comando

   ```
   python -m venv venv
   ```

   Ative o ambiente virtual com o comando

   ```
   source venv/bin/activate
   ```

   A prompt passa a ter algo como:

   ```
   (venv) user@machine:~/project$
   ```

   ⓘ A prompt com `(venv)` significa que:

   - `python` e `pip` passam a ser os do ambiente virtual
   - instalações ficam isoladas do sistema

   Cada terminal é independente — tens de ativar o ambiente em cada um.

3. **Instalação de dependências**

   corra o seu programa

   ```
   python manage.py runserver
   ```

   e instale os packages que ele for indicando em falta (tipicamente django, e Pillow) com o comando `pip install`. Exemplo:

   ```
   pip install django
   ```

   Quando a aplicação correr, quer dizer que tem instaladas todas as dependências.

4. **Guarde as dependências**

   Guarde as dependências que foram instaladas no ambiente virtual

   ```
   pip freeze > requirements.txt
   ```

ⓘ Se consultar o ficheiro criado, verificará que tem as dependências que instalou, e mais algumas dependências secundárias que precisa (por exemplo, o django precisa do `sqlparser`, entre outros)

Um repositório com `requirements.txt,` especifica as dependencias do projeto. Assim, quando quer trabalhar num projeto de um repositório do GitHub (cria um novo Codespace ou faz clone para o seu PC), os passos são sempre os mesmos:

1. criar ambiente virtual
2. ativar ambiente virtual
3. instalar de uma vez todas as dependencias com `pip install -r requirements.txt`

Super simples 😎!

Gitignore

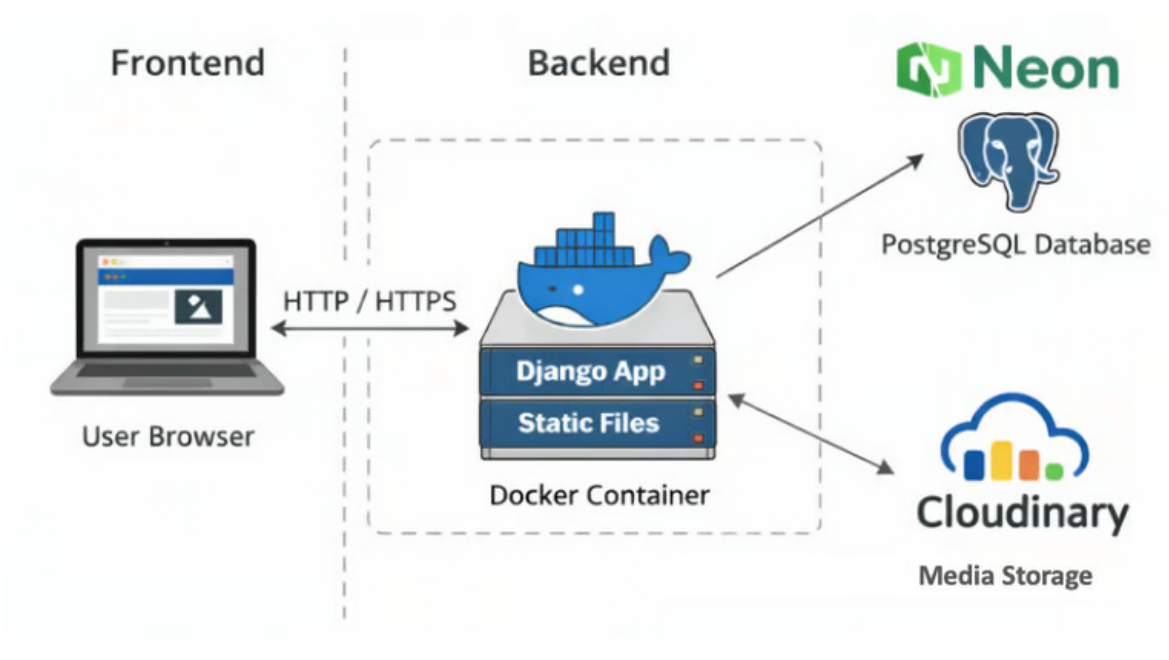
ⓘ `.gitignore` é um ficheiro de texto que instrui o Git sobre ficheiros ou pastas que devem ser ignorados no controle de versão. Localizado na raiz do repositório. Evita que ficheiros desnecessários ou sensíveis sejam versionados, mantendo o repositório limpo e seguro.

Crie o ficheiro `.gitignore` e inclua a pasta `venv/`, pois não pretende que seja versionada

```
## .gitignore

venv/
```

1. Migração de Base de Dados para Ambiente Cloud

Vamos agora criar uma base de dados PostgreSQL em Neon, e migrar os dados de db.sqlite para lá.


Frontend
Backend
Neon
PostgreSQL Database
HTTP / HTTPS
User Browser
Django App
Static Files
Docker Container
Cloudinary
Media Storage

1. Backup dos dados

Como vamos usar uma nova base de dados (BD), na cloud, e como já carregou dados na aplicação, exportamos os dados da BD atual para um ficheiro JSON:

```
python manage.py dumpdata app > dados.json
```

ⓘ **substituir `app` pelo nome da aplicação.**

Se tiver várias aplicações `app1`, `app2`, `app3`, especifique-as:

```
python manage.py dumpdata app1 app2 app3 > dados.json
```

Depois de executar este comando verá que aparece na sua pasta base o ficheiro `dados.json`. Se o abrir (formate-o com Ctrl + Shift + F), verá que tem os dados da sua base de dados.

2. Instalar Dependências

Instalar dependências necessárias:

```
pip install psycopg2-binary
python -m pip install dj-database-url
```

ⓘ
- `psycopg2-binary` → driver que permite ao Django comunicar com PostgreSQL
- `dj_databnase_url` → configura a base de dados a partir da variável `DATABASE_URL`

Atualizar o ficheiro `requirements.txt`:

```
pip freeze > requirements.txt
```

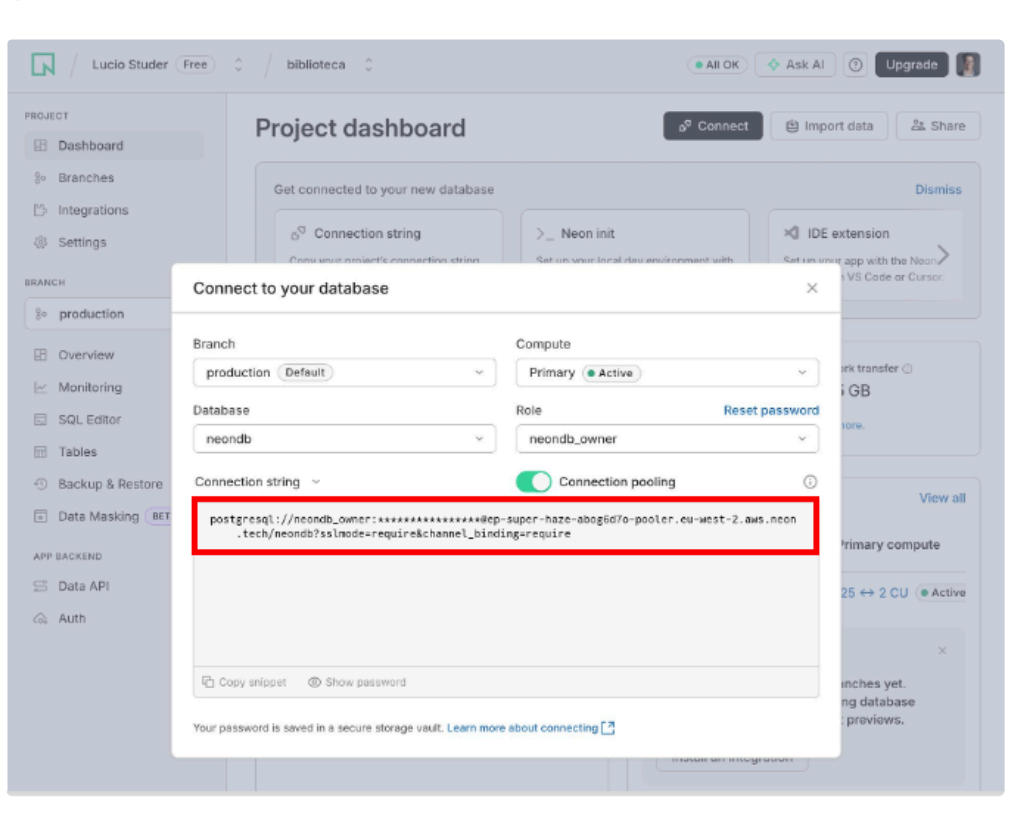
ⓘ `requirements.txt` ficará agora atualizado com ests duas novas dependências (abra-o para confirmar). Este ficheiro define todas as dependências necessárias para executar a aplicação em qualquer ambiente.

3. Criar base de dados PostgreSQL na Cloud (Neon)

ⓘ Neon é uma plataforma cloud que fornece bases de dados PostgreSQL serverless. Vamos usá-la para o nosso projeto.

Passos

1. Criar conta em Neon ↗ (use o seu github).
2. Criar um novo projeto para a sua base de dados
3. Copiar a **connection string** (disponível no separador *Neon/Dashboard*), clicar no botão **Connection string**)


Lucio Studer
Free
biblioteca
All OK
Ask AI
Upgrade
PROJECT
Dashboard
Branches
Integrations
Settings
BRANCH
production
Overview
Monitoring
SQL Editor
Tables
Backup & Restore
Data Masking
APP BACKEND
Data API
Auth
Project dashboard
Connect
Import data
Share
Get connected to your new database
Dismiss
Connection string
Neon init
IDE extension
Connect to your database
Branch
production
Default
Compute
Primary
Active
Database
neondb
Role
neondb_owner
Reset password
Connection string
Connection pooling
Copy snippet
Show password
Your password is saved in a secure storage vault. Learn more about connecting

```
postgresql://neondb_owner:****************@ep-super-haze-abog6d7o-pooler.eu-west-2.aws.neon
  .tech/neondb?sslmode=require&channel_binding=require
```

esta connection string tem toda a informação necessária para se ligar ªa base de dados: engine, nome, user, password

4. Variáveis de ambiente em .env

ⓘ Variáveis sensíveis são guardadas localmente no ficheiro .env, garantindo segurança e portabilidade. Podem depois ser usadas no código.

1. No ambiente local, usamos o ficheiro `.env` para definir a variável `DATABASE_URL` com a *connection string* do Neon, uma chave que deve permanecer secreta:

```
DATABASE_URL=postgresql://username:password@db.project.com:5432/myapp?sslmode=require
abase
```

ⓘ substituir `username`, `password` e `host` pelos dados fornecidos pelo Neon. Sem espaços em torno de `=`.

2. O ficheiro `.env` **não deve ser incluído no repositório**. Inclua-o em `.gitignore`:

```
## .gitignore

venv/
.env
```

5. Configurar Django (settings.py)

Em `project/settings.py`, ligue a aplicação à nova base de dados, definindo a variável `DATABASES`

Primeiro, instale a dependencia que permite inserir variáveis de ambiente num determinado ficheiro, e atualize o `requirements.txt`

```
pip install django-environ
pip freeze > requirements.txt
```

Agora, em `settings.py`, defina a variável `DATABASES` da seguinte forma:

```
## em settings.py

import environ
import os

# inicializar environ
env = environ.Env()

# ler ficheiro .env (opcional mas recomendado)
environ.Env.read_env(os.path.join(BASE_DIR, ".env"))

## definicao da base de dados psql em Neon
DATABASES = {
    "default": env.db("DATABASE_URL")
}
```

6. Inicialização e validação da base de dados

Vamos criar uma base de dados, com as entidades que defin estrutura da base de dados passa de vazia → pronta a usar

Se correr a aplicação, verá que tem erros. Isto porque não tem a base de dados configurada. Deverá, na consola com o ambiente virtual ativado, executar:

1. **Criar estrutura da BD**

   Crie tabelas vazias da sua aplicação no Neon

```
python manage.py migrate
```

2. **Carregar dados**

   Carregue os dados, repondo o estado anterior

```
python manage.py loaddata dados.json
```

3. **Criar Superuser**

   Crie um utilizador administrador

```
python manage.py createsuperuser
```

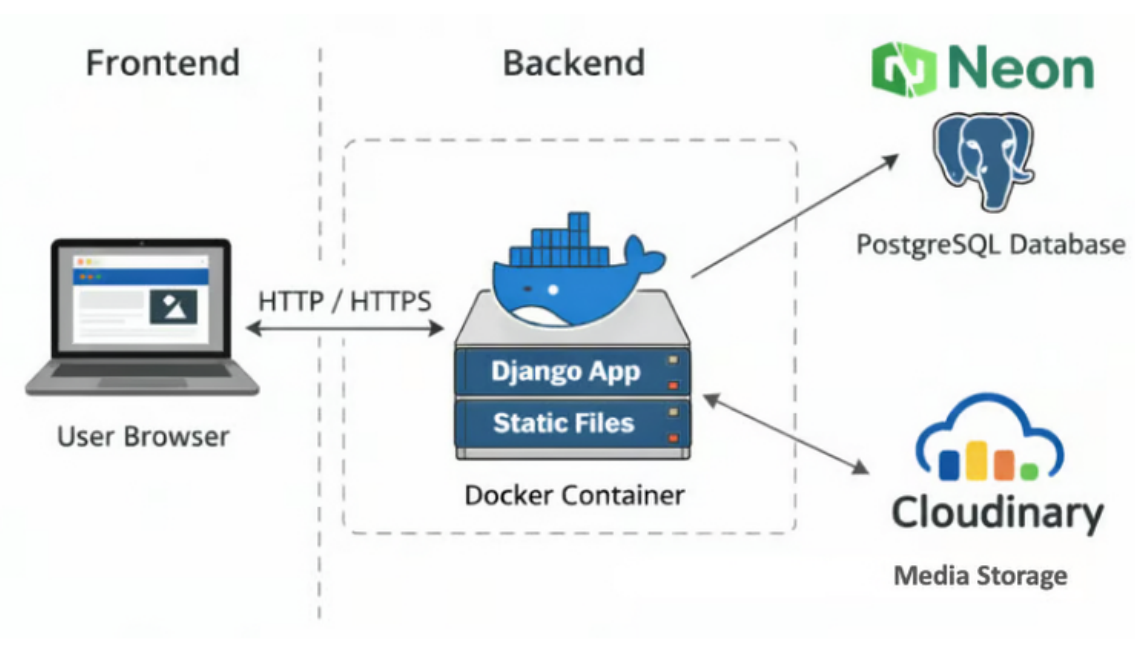
4. **Validar (Admin / Neon UI)**

   Pode visualizar e gerir os dados através de:

   - Django Admin (`/admin`)
   - Interface web do Neon, clique em Tables para ver as tabelas, e seus valores.

2. Migração de ficheiros de media para Cloudinary

Objetivo: vamos agora migrar o armazenamento local de ficheiros da pasta local (media/) para a cloud usando Cloudinary.


Frontend
Backend
Neon
User Browser
HTTP / HTTPS
Django App
Static Files
Docker Container
PostgreSQL Database
Cloudinary
Media Storage

1. Estado inicial (armazenamento local)

Em modelações que usem o armazenamento de imagens e ficheiros, o armazenamento é feito localmente numa pasta `media` ou `mediafiles`.

Em `project/settings.py` terá tipicamente:

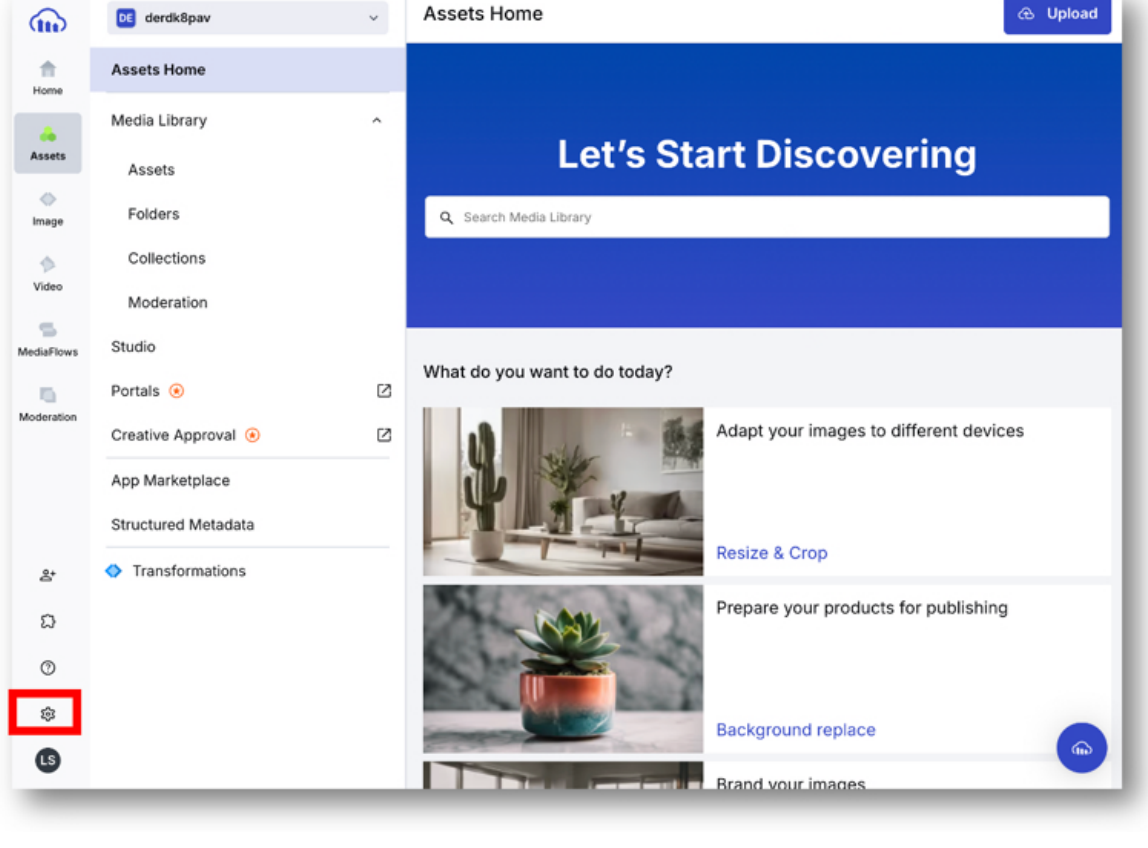
```
## project/settings.py

STORAGES = {
    "default": {
        "BACKEND": "django.core.files.storage.FileSystemStorage",
    },
    "staticfiles": {
        "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage",
    },
}

MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'mediafiles')
```

No ficheiro `project/urls.py` deverá ter

```
## project/urls.py

##...

from django.conf import settings
from django.conf.urls.static import static

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

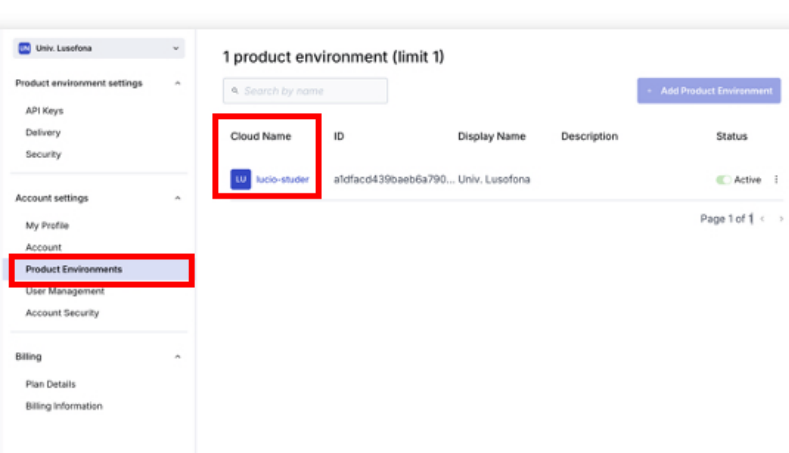
2. Criar conta no Cloudinary

1. Aceder a https://cloudinary.com/ ↗
2. Criar conta gratuita (use o seu github)
3. No dashboard, em *__Settings__* (em baixo)


derdk8pav
Assets Home
Media Library
Assets
Folders
Collections
Moderation
Studio
Portals
Creative Approval
App Marketplace
Structured Metadata
Transformations
Home
Assets
Image
Video
MediaFlows
Moderation
Assets Home
Upload
Let's Start Discovering
Search Media Library
What do you want to do today?
Adapt your images to different devices
Resize & Crop
Prepare your products for publishing
Background replace
Brand your images

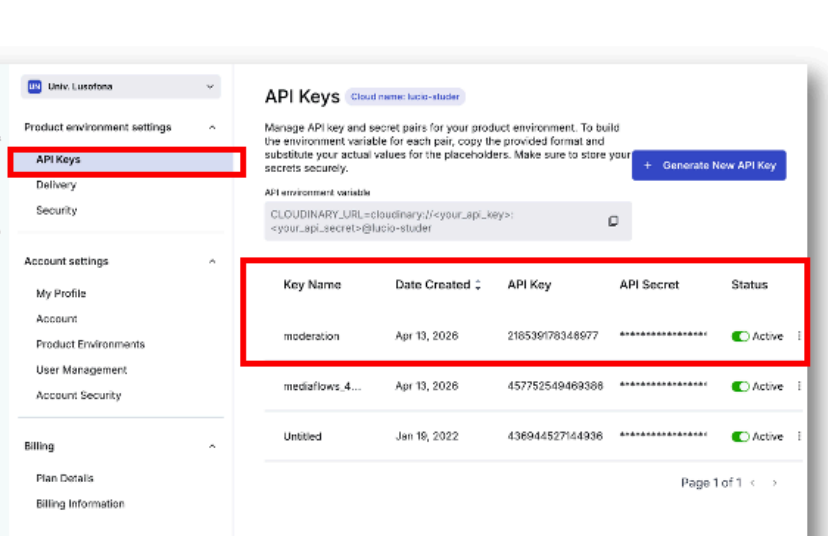
1. Em ***Account settings/Product Environment***, copiar para a aplicação Notepad:

- Cloud Name (Account settings/disponivel em product environment)


Univ. Lusófona
Product environment settings
API Keys
Delivery
Security
Account settings
My Profile
Account
Product Environments
User Management
Account Security
Billing
Plan Details
Billing Information
1 product environment (limit 1)
Add Product Environment
Cloud Name
ID
Display Name
Description
Status
lucio-studer
Univ. Lusofona
Active
Page 1 of 1

5. Em **Product environment settings/API Keys** copiar

- API Key
- API Secret (para ver o Secret, irá receber um email com um codigo que ativa a visualização)


Univ. Lusófona
Product environment settings
API Keys
Delivery
Security
Account settings
My Profile
Account
Product Environments
User Management
Account Security
Billing
Plan Details
Billing Information
API Keys
Generate New API Key
Key Name
Date Created
API Key
API Secret
Status
moderation
mediaflows_4...
Untitled
Active
Page 1 of 1

3. Instalar dependências

Instale dependencias para trabalhar com cloudinary, e atualize o `requirements.txt`

```
python -m pip install cloudinary django-cloudinary-storage
pip freeze > requirements.txt
```

4. Configurar o Django

1. **Adicionar apps**

```
## settings.py

INSTALLED_APPS = [
    ...
    'cloudinary',
    'cloudinary_storage',
]
```

2. **Variáveis de ambiente**

   1. Em `.env` adicionar as variáveis de ambiente e valores que copiou de Cloudinary

```
CLOUDINARY_CLOUD_NAME=xxxxx
CLOUDINARY_API_KEY=xxxxx
CLOUDINARY_API_SECRET=xxxxx
```

   2. Configurar credenciais do Cloudinary em `settings.py`

```
## settings.py

import environ
import os

# já foi feita a importação para as var. de ambiente da BD
env = environ.Env()
environ.Env.read_env(os.path.join(BASE_DIR, ".env"))

CLOUDINARY_STORAGE = {
    'CLOUD_NAME': env('CLOUDINARY_CLOUD_NAME'),
    'API_KEY': env('CLOUDINARY_API_KEY'),
    'API_SECRET': env('CLOUDINARY_API_SECRET'),
}
```

3. **Ativar Cloudinary como storage**

```
# settings.py

STORAGES = {
    "default": {
        "BACKEND": "cloudinary_storage.storage.MediaCloudinaryStorage",
    },
    "staticfiles": {
        "BACKEND": "django.contrib.staticfiles.storage.StaticFilesStorage",
    },
}
```

6. Migrar ficheiros existentes

Os ficheiros atuais associados à base de dados estão na pasta `media/` (ou `mediafiles/`) e precisam de ser enviados para a cloud. Para cada objeto queremos:

- ler o ficheiro local (em `media/`)
- fazer upload para Cloudinary
- substituir URL armazenado na base de dados

1. **Script de migração**

   Crie um script que carregue cada imagem. A aplicação já está configurada para guardar no CLoudinary. Para cada classe que tem uma imagem, tem que obter o objeto, e atualizar o ficheiro associado indo buscá-lo à pasta media onde ainda está.

   Eis um exemplo de ficheiroo script `migra_ficheiros.py`. Adeque com cuidado o código ao modelo. Deverá criar um ficheiro para cada classe de models que tem uma imagem.

```
import os
from django.core.files import File
from escola.models import Curso   # adaptar ao modelo

for obj in Curso.objects.all():
    if obj.imagem and obj.imagem.name:   # adaptar o nome do campo (neste caso é "imag
        local_path = obj.imagem.path    # adequar

        if os.path.exists(local_path):
            with open(local_path, 'rb') as f:
                obj.imagem.save(                         # adequar
                    os.path.basename(local_path),
                    File(f),
                    save=True
                )
            print(f"Migrado: {obj}")
```

ⓘ O que o script faz:

O que o script faz:

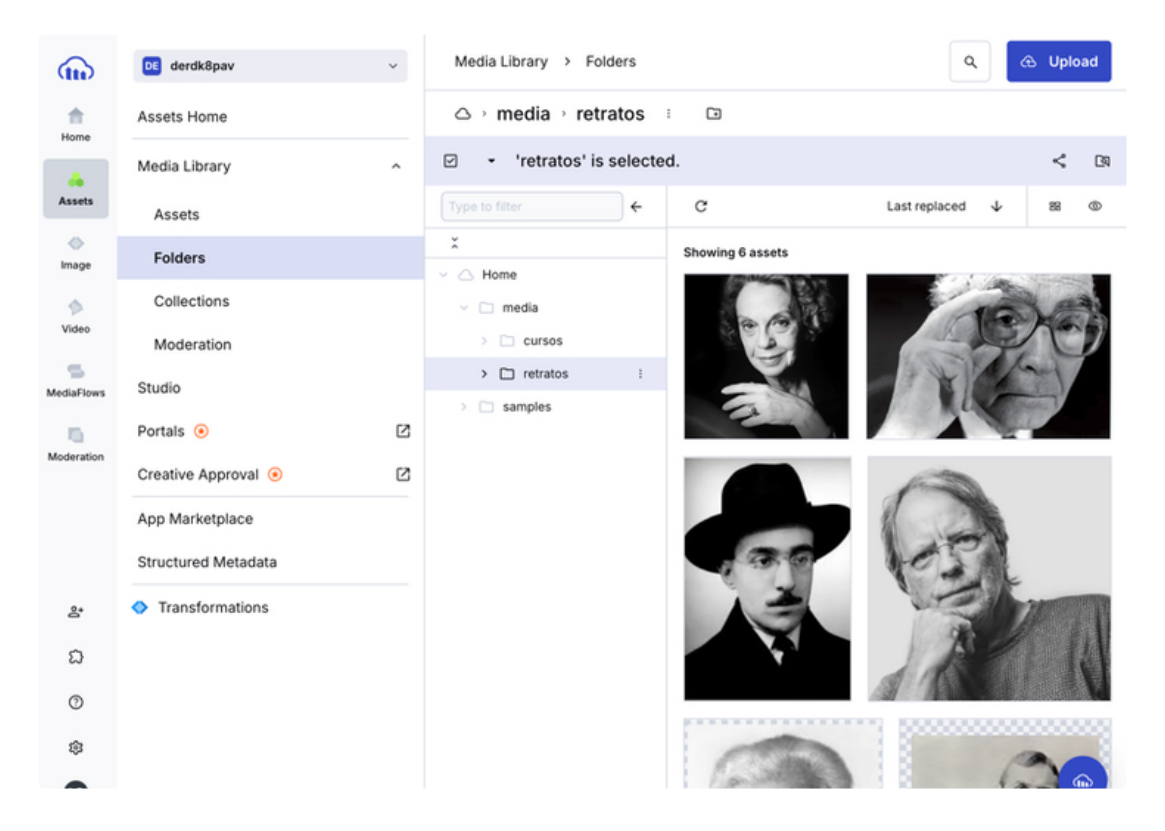
- lê ficheiros locais (`media/`)
- volta a guardar no modelo
- o Django usa Cloudinary automaticamente
- o ficheiro passa a estar na cloud

2 **Executar**

```
python manage.py shell
>>> import migra_ficheiros
```

3 **Verificar**

verifique se foi criada a pasta com o nome da classe


derdk8pav
Assets Home
Media Library
Assets
Folders
Collections
Moderation
Studio
Portals
Creative Approval
App Marketplace
Structured Metadata
Transformations
Media Library > Folders
Upload
media > retratos
'retratos' is selected.
Type to filter
Last replaced
Home
media
cursos
retratos
samples
Showing 6 assets

7. Testar

1. Criar ou editar um objeto no admin
2. Fazer upload de uma imagem
3. Confirmar que a URL é do tipo:

```
https://res.cloudinary.com/...
```

8. Limpeza final

Depois de confirmar que tudo funciona:

❌ **Remover configuração local**

```
# MEDIA_URL = ...
# MEDIA_ROOT = ...
```

❌ **Remover suporte a media no `urls.py`**

```
# if settings.DEBUG:
#     urlpatterns += static(...)
```

3. Ficheiros estáticos

ⓘ **CSS, JS, logotipos** e imagens[1] do site, são **ficheiros estáticos**

[1]: não inclui upload de imagens por utilizadores

Num ambiente em produção, para servir **ficheiros estáticos** usamos ***WhiteNoise***, uma biblioteca Python que permite ao Django servir ficheiros estáticos de forma eficiente.

Instale o whitenoise no terminal com o ambiente virtual ativad

```
pip install whitenoise
pip freeze > requirements.txt
```

Como pretendemos preparar o nosso projeto para produção, adicionamos:

```
## settings.py

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",    # <-- 1. adicione
     ## ...
]

STORAGES = {
    "staticfiles": {                               # <-- 2. adicione
        "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage",
    },
}

MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'mediafiles')    # <-- 3. adicione
```

Sempre que alterar algum ficheiro estático deverá executar

python manage.py collectstatic

4. App Django a correr num Container Docker

No GitHub

Nas próximas secções:

1. irá criar um repositório GitHub na organização da disciplina de Programação Web. Este repositório será baseado num *template* que já inclui um conjunto de componentes necessários para executar o seu projeto num *container* Docker, no servidor do DEISI.
2. Irá depois unir o código do seu projeto (no seu repositório GitHub que criou inicilamente) com o conteúdo do novo repositório template. Com comandos adequados, o Git irá combinar os dois conjuntos de ficheiros — e como não há nomes repetidos, tudo corre bem sem conflitos.

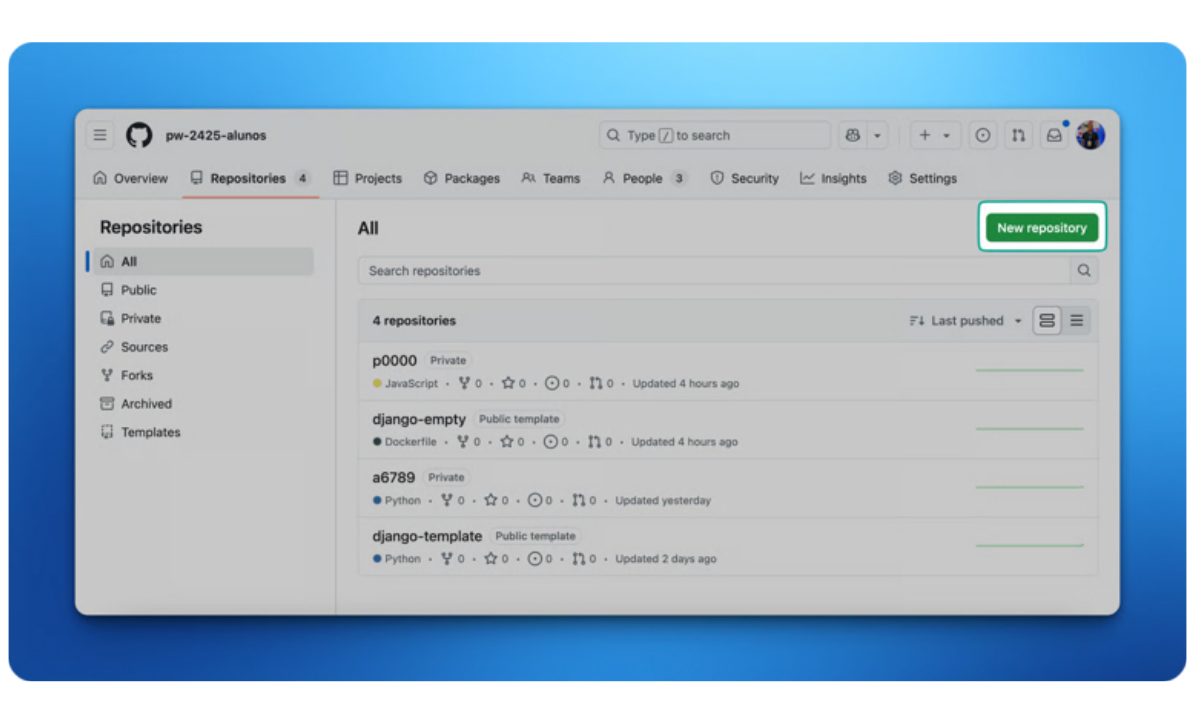
1. Organização

Fale com o seu professor para ser convidado para fazer da organização de Programação Web no GitHub.

[Programação Web 25/26 AlunosGitHub ↗](#)

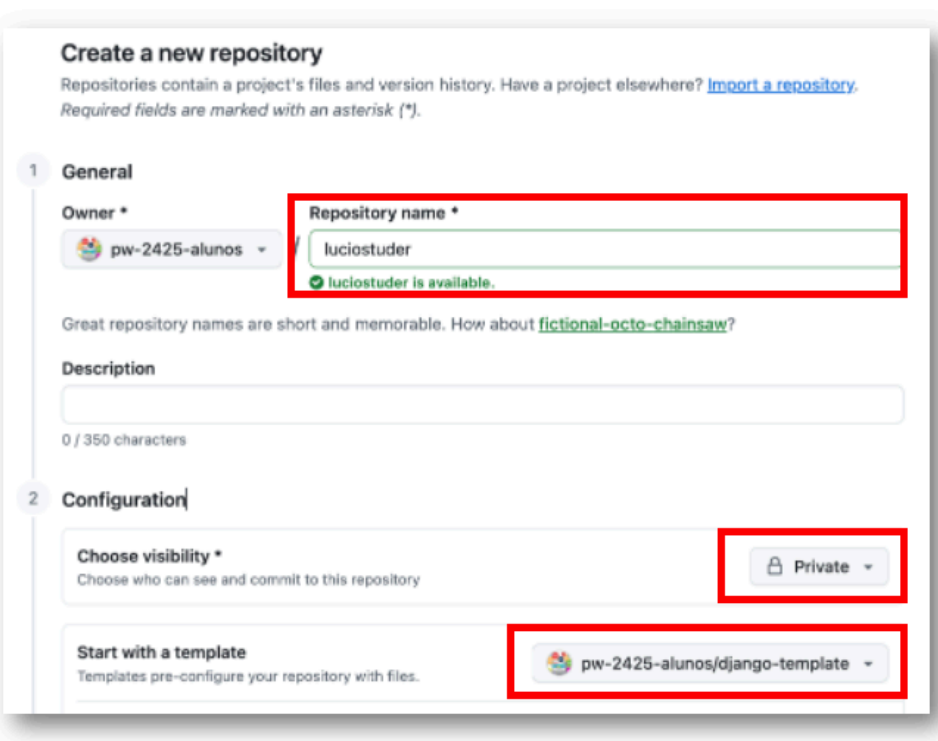
2. Criar Repositório

1. Crie um repositório privado em: [https://github.com/organizations/pw-2425-alunos/repositories/new ↗](https://github.com/organizations/pw-2425-alunos/repositories/new)


pw-2425-alunos
Overview
Repositories 4
Projects
Packages
Teams
People 3
Security
Insights
Settings
Repositories
All
Public
Private
Sources
Forks
Archived
Templates
New repository
Search repositories
4 repositories
Last pushed
p0000 Private
django-empty Public template
a6789 Private
django-template Public template

2. Configuração do repositório:

⚠
1. Deve usar o **django-template** como template. Este tem um template com CI/CD.
2. Como nome use o seu nome e apelido, seguido de numero de aluno.
3. O repositório deve ser privado



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).

1 General

Owner * pw-2425-alunos / Repository name * luciostuder

luciostuder is available.

Great repository names are short and memorable. How about fictional-octo-chainsaw?

Description

0 / 350 characters

2 Configuration

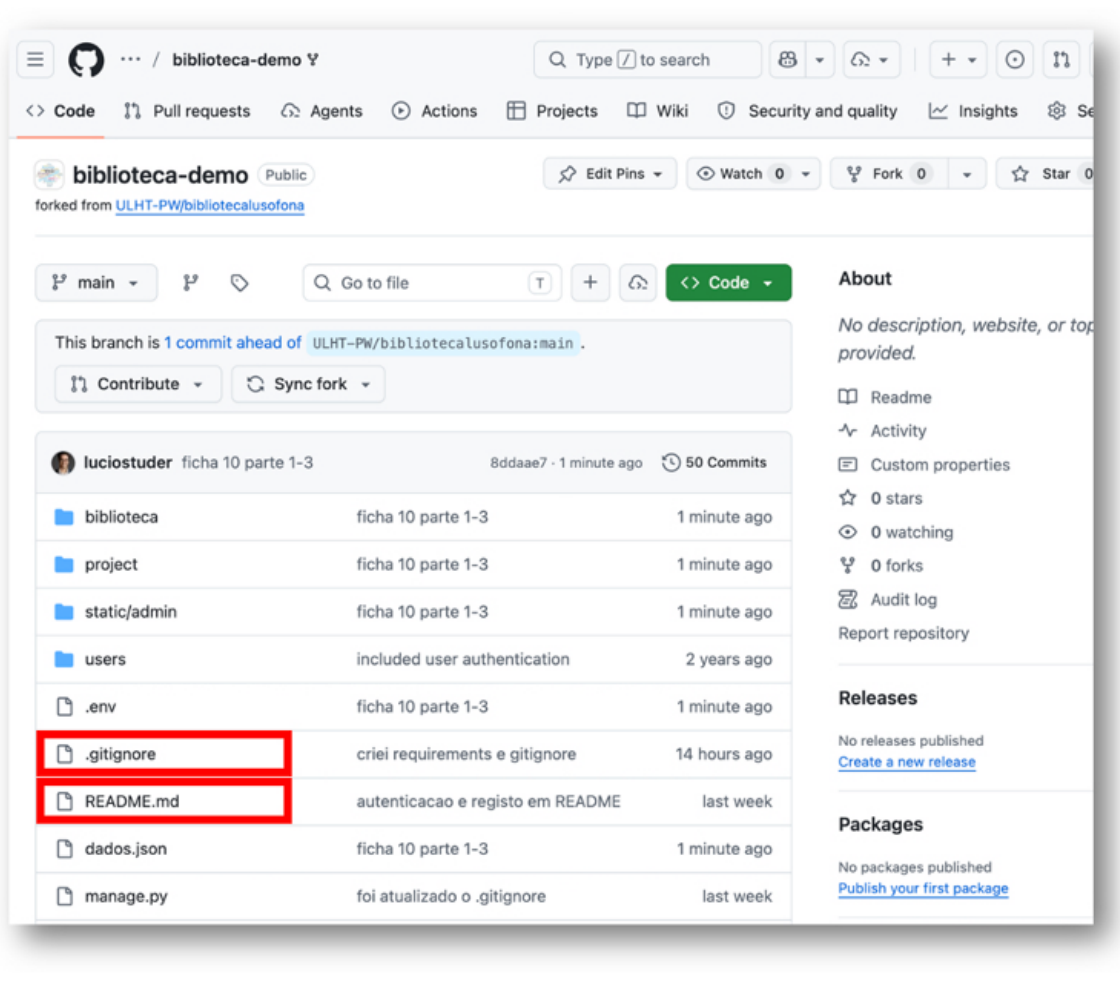
Choose visibility * — Choose who can see and commit to this repository: Private

Start with a template — Templates pre-configure your repository with files.: pw-2425-alunos/django-template

3. Preparativos: evitar conflitos de ficheiros

Irá agora fazer uma junção de repositorios. Devemos evitar conflitos de ficheiros com o mesmo nome. Se observar o repositorio que criou agora, tem um ficheiro README.md e .gitignore

No repositorio do seu portfolio eventualmente tambem existem esses ficheiros.


biblioteca-demo
Code
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Public
Edit Pins
Watch 0
Fork 0
Star
main
Go to file
Code
Contribute
Sync fork
About
Readme
Activity
Custom properties
0 stars
0 watching
0 forks
Audit log
Report repository
Releases
Packages

forked from ULHT-PW/bibliotecalusofona

This branch is 1 commit ahead of `ULHT-PW/bibliotecalusofona:main`.

luciostuder ficha 10 parte 1-3	8ddaae7 · 1 minute ago	50 Commits
biblioteca	ficha 10 parte 1-3	1 minute ago
project	ficha 10 parte 1-3	1 minute ago
static/admin	ficha 10 parte 1-3	1 minute ago
users	included user authentication	2 years ago
.env	ficha 10 parte 1-3	1 minute ago
.gitignore	criei requirements e gitignore	14 hours ago
README.md	autenticacao e registo em README	last week
dados.json	ficha 10 parte 1-3	1 minute ago
manage.py	foi atualizado o .gitignore	last week

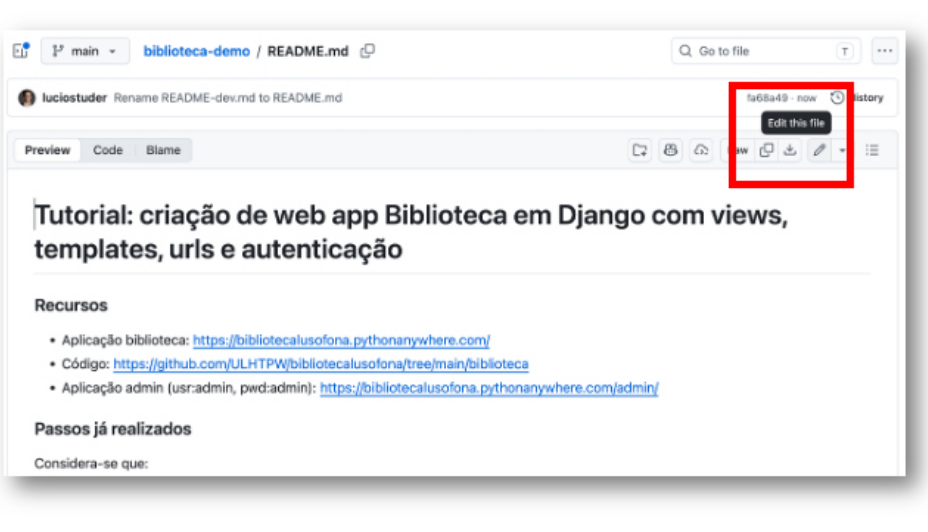
No description, website, or topics provided.

No releases published
Create a new release

No packages published
Publish your first package

Para evitar conflitos, diretamente no GitHub (não no codespaces), no repositorio do seu projeto:

1. Se tiver um ficheiro README.md, modifique o seu nome para README-dev.md.


main
biblioteca-demo / README.md
luciostuder Rename README-dev.md to README.md
Edit this file
Preview
Code
Blame

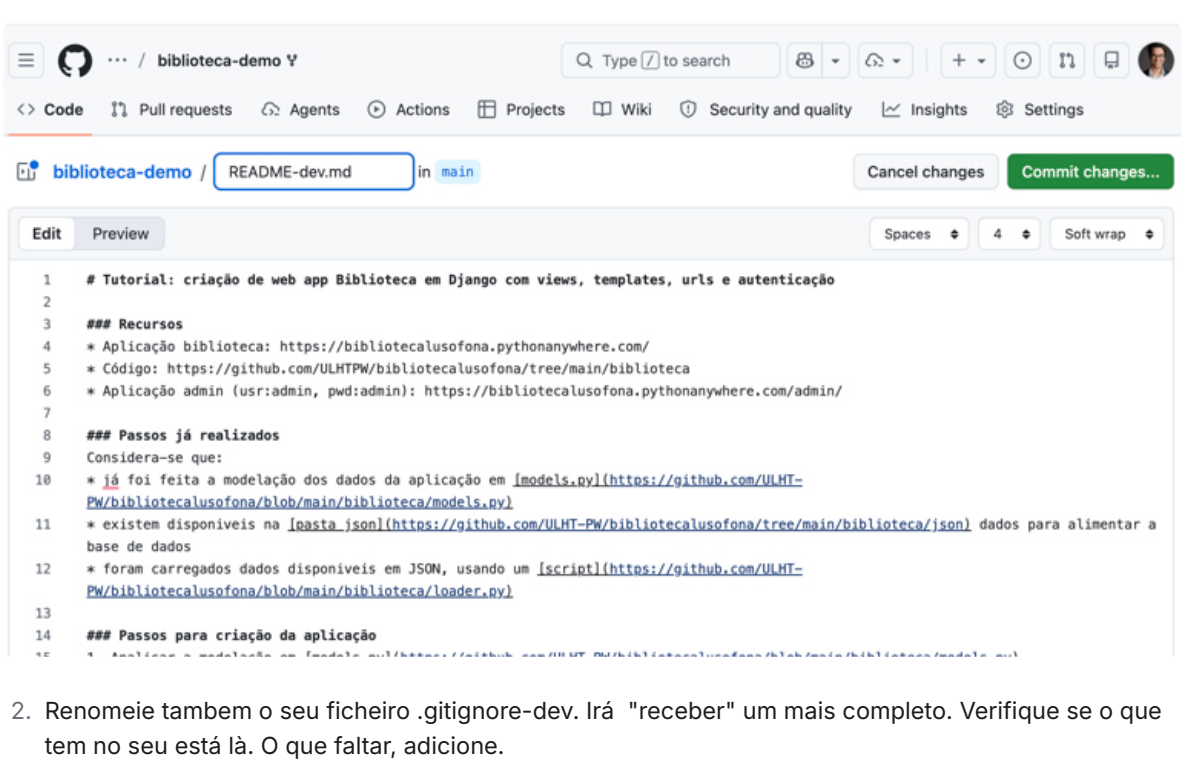
Tutorial: criação de web app Biblioteca em Django com views, templates, urls e autenticação

Recursos

- Aplicação biblioteca: https://bibliotecalusofona.pythonanywhere.com/
- Código: https://github.com/ULHTPW/bibliotecalusofona/tree/main/biblioteca
- Aplicação admin (usr:admin, pwd:admin): https://bibliotecalusofona.pythonanywhere.com/admin/

Passos já realizados

Considera-se que:


biblioteca-demo
Code
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Cancel changes
Commit changes...
Edit
Preview
Spaces
4
Soft wrap

biblioteca-demo / README-dev.md in main

```
# Tutorial: criação de web app Biblioteca em Django com views, templates, urls e autenticação

### Recursos
* Aplicação biblioteca: https://bibliotecalusofona.pythonanywhere.com/
* Código: https://github.com/ULHTPW/bibliotecalusofona/tree/main/biblioteca
* Aplicação admin (usr:admin, pwd:admin): https://bibliotecalusofona.pythonanywhere.com/admin/

### Passos já realizados
Considera-se que:
* já foi feita a modelação dos dados da aplicação em [models.py](https://github.com/ULHT-PW/bibliotecalusofona/blob/main/biblioteca/models.py)
* existem disponiveis na [pasta json](https://github.com/ULHT-PW/bibliotecalusofona/tree/main/biblioteca/json) dados para alimentar a base de dados
* foram carregados dados disponiveis em JSON, usando um [script](https://github.com/ULHT-PW/bibliotecalusofona/blob/main/biblioteca/loader.py)

### Passos para criação da aplicação
```

2. Renomeie tambem o seu ficheiro .gitignore-dev. Irá "receber" um mais completo. Verifique se o que tem no seu está là. O que faltar, adicione.

Dê commit ao seu projeto, de modo a que estas alterações estejam no GitHub.

4. Carregar o Código

Neste secção vamos passar o código do seu projeto para este novo repositório.

1. Abra com codespaces o seu novo repositorio em [https://github.com/pw-2425-alunos/ ↗](https://github.com/pw-2425-alunos/). No terminal execute

```
git clone https://github.com/seu-username/seu-repo
```

onde o link é do repositório do seu portfolio. O repositório aparecerá numa pasta que designaremos aqui de `novo-repo`.

3. Se tiver um ficheiro README.md, modifique o seu nome para README-projeto.md.
4. Renomeie tambem o seu ficheiro .gitignore-dev. Irá "receber" um mais completo. Verifique se o que tem no seu está là. O que faltar, adicione.
5. Com o comando em baixo, copie o conteúdo do projeto em `novo-repo` para a pasta principal, de modo a que o `manage.py` esteja na mesma pasta que o `.gitignore` e `docker-compose.yml`. Para tal, no mesmo terminal, execute (onde `novo-repo`. é o nome da sua nova pasta)

```
cp -r novo-repo/* .
```

ⓘ com o * não copiamos ficheiros ocultos, que sao o git do repo que nao queremos usar.

4. Apague agora a pasta `novo-repo`.
5. com os comandos em baixo, crie um ambiente virtual, ative-o, instale as dependencias e corra a aplicação para ver se está tudo bem:

```
$ python -m venv venv
$ source venv/bin/activate
(venv) $ pip install -r requirements.txt
(venv) $ python manage.py runserver
```

5. Atualizar requirements.txt

adicionar a `requirements.txt, diretamente n oficheiro e numa linha`

```
gunicorn
```

Garanta que `requirements.txt` tem pelo menos as seguintes dependências:

```
Django
cloudinary
django-cloudinary-storage
django-environ
dj-database-url
gunicorn
pillow
psycopg2-binary
whitenoise
```

6 Variáveis de ambiente em .env

Variáveis sensíveis são guardadas localmente em .env e, em produção, como Secrets no GitHub, garantindo segurança e portabilidade.

Como `.env` está listado no `.gitignore`, não está no repo que clonámos (ficou no codespaces).

Crie novamente um ficheiro `.env` e coloque as variáveis que tem no seu repositorio original do portfolio.

Deverá adicionar mais variáveis. Em particular, de `settings.py` as variaveis `DEBUG`, `SECRET_KEY`, e as credenciais da conta de email, caso tenha implementado o magic link.

Em **desenvolvimento**, o Django deve estar em modo debug ativo, ou seja:

- Mostra páginas de erro detalhadas
- Recarrega automaticamente o servidor sempre que faça alterações ao código
- Mostra informações úteis para programar e depurar

Num ambiente de produção deve ter `DEBUG = False`, porque o modo debug pode expor informação sensível da aplicação.

Valide se a aplicação continua a correr bem.

7 Variáveis de ambiente para produção em GitHub Secrets

Variáveis sensíveis, em produção, são guardadas como Secrets no GitHub, garantindo a mesma segurança.

Em produção, não usamos `.env`. Em vez disso, usamos **Secrets no GitHub**.

> Os *Secrets* no GitHub são variáveis seguras usadas para guardar informações sensíveis, como passwords, chaves de API ou tokens, fora do código-fonte. Estes dados ficam encriptados e só são acessíveis durante processos como deploys ou workflows, evitando a sua exposição pública no repositório.

Como configurar Secrets no GitHub:

1. Ir ao repositório no GitHub
2. Clicar em **Settings**
3. No menu lateral:
    - **Secrets and variables → Actions**
4. Clicar em **New repository secret.** Adicionar variável, definindo:

- **Name:** `DATABASE_URL`
- **Value:**

```
postgresql://username:password@db.project.com:5432/myapp?sslmode=require
```

5. Clicar em **Add secret**

Crie Repository Secrets para as restantes variáveis que definiu em .env:

5. `DEBUG`
6. `DATABASE_URL`
7. `CLOUDINARY_CLOUD_NAME`
8. `CLOUDINARY_API_KEY`
9. `CLOUDINARY_API_SECRET`

8. Aplicação publicada 🌤️

Vamos aqui ver a imagem Docker em execução em produção

Nesta secção irá publicar e ver a correr em produção a sua aplicação

Publicação

Faça commit das alteracoes que fez

```
git add .
git commit -m "migração do projeto Django no template com CI/CD"
git push
```

Deploy

Agora de forma automatica, o GitHub Actions irá dar deploy do seu projeto.

Irá verificar que o seu projeto ficou automáticamente live **em modo produção no servidor do DEISI**, a correr **num container docker**.

O *template* usado `django-empty` já tem ficheiros de configuração para o pipeline CI/CD (em **.github/workflows**) que automatiza o *build*, *push* e *deploy* da imagem Docker no servidor do DEISI. O seu projeto passa a estar num ambiente de produção.

Verifique que seu website se encontra automáticamente live em:

```
https://nomedorepo.pw.deisi.ulusofona.pt/
```

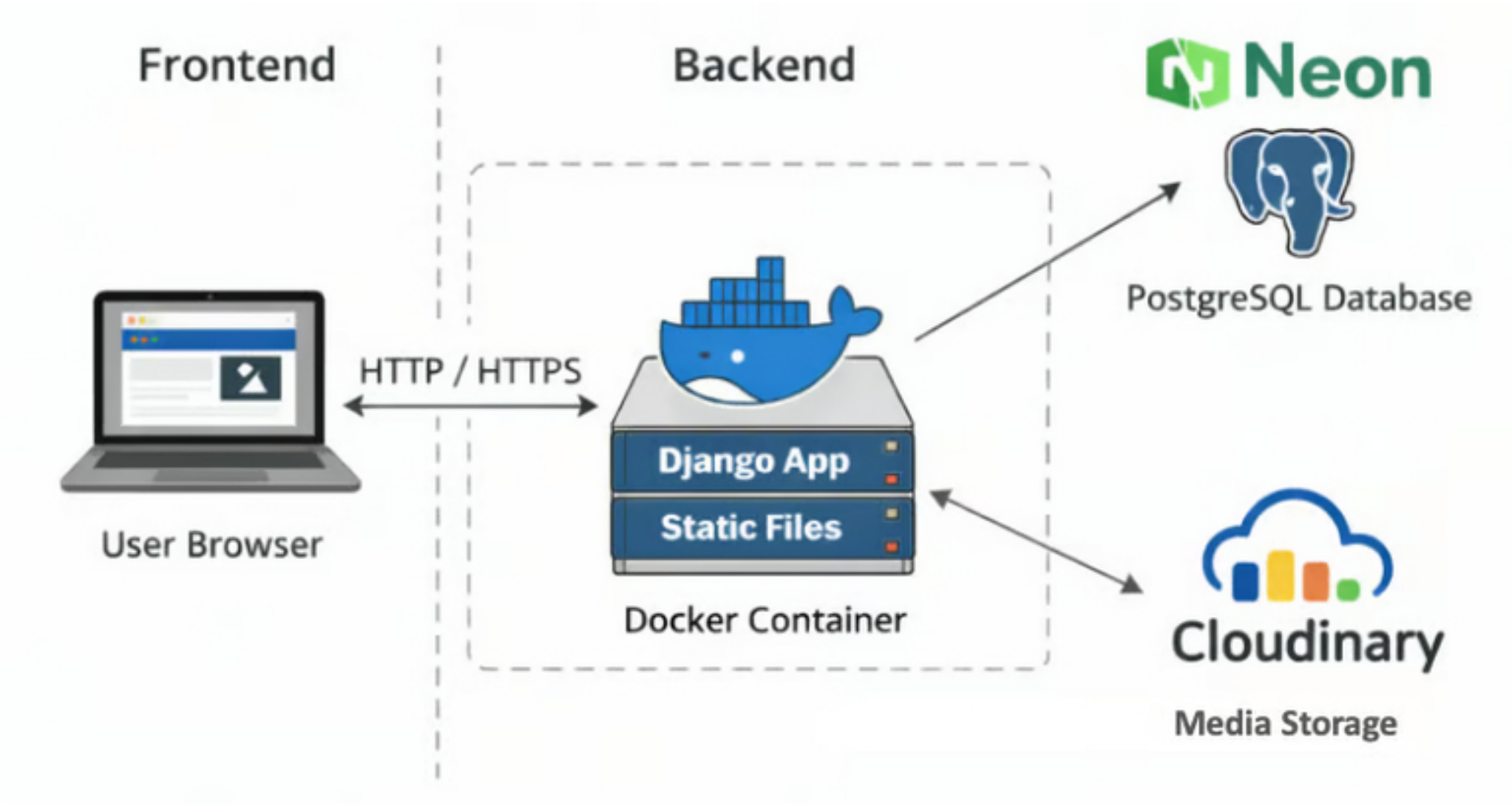
🥳 a sua app está live na cloud 🌤️

> Altere o `nomedorepo` para o nome do seu repositório

> demora uns minutos a ficar live.

A sua web app portfolio encontra-se online, usando uma moderna:

1. base de dados PostgreSQL, alojada no Neon
2. storage de ficheiros, alojados no Cloudinary
3. alojamento da aplicação num container a correr num servidor da lusófona



Os três passos sao independentes. Terá sempre a sua aplicação a funcionar ao longo da sua implementação.

9. Correr num servidor, num container Docker?

🐳 O que significa "correr num servidor num container Docker"?

Quando dizemos que um projeto está "a correr num container Docker num servidor", significa o seguinte:

- **Docker** é uma tecnologia que permite empacotar uma aplicação com tudo o que ela precisa para funcionar: código, dependências, configurações, etc.
- Este pacote é chamado de **imagem Docker**, e quando é executado, cria-se um **container** — uma espécie de miniambiente isolado que funciona como se fosse um pequeno computador, mas dentro do servidor.

👉 **É como enviar a sua aplicação numa caixinha que pode ser executada em qualquer lugar, sem depender da configuração do servidor.**

⚙️ O que torna isso possível?

Tudo isto funciona automaticamente graças a três componentes principais:

1. **GitHub Actions (CI/CD)** – Quando faz um *push* para o GitHub, os ficheiros dentro de `.github/workflows` iniciam um processo automático (pipeline) que:
    - constrói a imagem Docker da sua aplicação,
    - envia (faz *push*) essa imagem para o servidor do DEISI,
    - e executa (*deploy*) essa imagem num container.
2. **Servidor do DEISI com Docker** – Está preparado para receber imagens Docker e executá-las, alojando-as num endereço web próprio.
3. **Nomenclatura do repositório** – O nome do seu repositório (por exemplo, `a2100000`) é usado para gerar automaticamente o endereço onde a aplicação ficará disponível.

Only this page
All pages
Powered by GitBook